

Resiliencia, Manejo de Límites y LLM

Estrategias de robustez en Sentinel AcademIA

Circuit Breaker, Fallback, Rate Limiting, DLQ, Idempotencia y Observabilidad

Equipo de Desarrollo
sentinel-academia@universidad.edu

20 de junio de 2026

Resumen Ejecutivo

Este documento detalla las **9 estrategias de resiliencia** implementadas en Sentinel AcademIA para garantizar la robustez del sistema ante fallos de la IA generativa, picos de carga, errores externos y problemas de red. La arquitectura está diseñada para **degradar de forma elegante** (de Cohere Command R → Llama 3.1 → Command R+ → respuesta controlada), mantener **idempotencia exactly-once** en consumers SQS, y proporcionar **observabilidad end-to-end** con CloudWatch, X-Ray y Langfuse. El sistema cumple SLA objetivo de **99% de éxito en procesamiento y detección de casos críticos en < 1 minuto**.

Índice

1. Introducción: Por qué la resiliencia es crítica	3
1.1. Las 9 estrategias implementadas	3
2. Timeouts en Llamadas Externas	3
2.1. Configuración estándar	3
2.2. Timeouts por servicio	4
3. Circuit Breaker con opossum	4
3.1. Qué es y por qué lo necesitamos	5
3.2. Implementación con opossum	5
3.3. Métricas del circuit breaker	6
4. Fallback Chain Multi-Modelo	6
4.1. Cadena de modelos	6
4.2. Implementación del router con fallback	7
4.3. Justificación de cada modelo	8
5. Rate Limiting con Token Bucket	8
5.1. Algoritmo y justificación	8
5.2. Integración en handler	9
5.3. Configuración por endpoint	10

6. Retry con Exponential Backoff y Jitter	10
6.1. Cuándo reintentar	10
6.2. Implementación con p-retry	10
6.3. Backoff exponencial + jitter	11
7. Idempotencia Exactly-Once Processing	11
7.1. El problema	11
7.2. La solución: SHA-256 + conditional writes	11
8. Dead Letter Queues (DLQ)	12
8.1. Obligatorias en cada cola	12
8.2. SAM template para DLQ	13
9. Validación del Output del LLM con Zod	14
9.1. El problema de los LLMs	14
9.2. Schema Zod estricto	14
9.3. Funciones de extracción y validación	14
10. Logging Estructurado	15
10.1. Formato JSON obligatorio	15
10.2. Correlation ID end-to-end	16
11. Observabilidad End-to-End	16
11.1. CloudWatch Logs Insights queries	16
11.2. Langfuse para tracing del LLM	17
12. Costos y Monitoreo de LLM	18
12.1. Tracking de tokens	18
12.2. Estimación de costos	18
13. Diagrama de Flujo Completo de Resiliencia	19
14. Resultados Esperados y SLAs	19
14.1. SLAs objetivo	19
14.2. Cumplimiento de la rúbrica	19

1 Introducción: Por qué la resiliencia es crítica

Sentinel AcademIA depende de múltiples servicios externos y distribuidos:

- **Oracle OCI Generative AI:** provee el LLM, puede tener rate limits o caídas
- **AWS SQS / DynamoDB / S3:** servicios managed pero no 100 % disponibles
- **Red cross-region:** latencia variable entre AWS y OCI (~80ms)
- **Frontend Vercel:** deploy en edge, posibles problemas de cache

Sin estrategias de resiliencia, una caída de OCI GenAI de 5 minutos significaría perder el análisis de cientos de quejas, incluyendo las críticas. **Esto es inaceptable** para un sistema que detecta casos de salud mental o acoso.

1.1 Las 9 estrategias implementadas

1. **Timeouts** en TODA llamada externa
2. **Circuit Breaker** con opossum
3. **Fallback chain** entre 3 modelos LLM
4. **Rate limiting** con token bucket
5. **Retry** con exponential backoff y jitter
6. **Idempotencia** con SHA-256 y conditional writes
7. **Dead Letter Queues** en cada cola SQS
8. **Validación Zod** del output del LLM
9. **Observabilidad** con CloudWatch, X-Ray, Langfuse

Principio fundamental

“Diseñar para el fallo, no para el camino feliz.” Toda llamada externa PUEDE fallar. Toda respuesta PUEDE llegar mal. Todo servicio PUEDE caerse. El sistema debe seguir operando de forma degradada.

2 Timeouts en Llamadas Externas

2.1 Configuración estándar

Toda llamada a un servicio externo tiene un timeout explícito. **Nunca** se usa el default del SDK (que puede ser de minutos o inexistente).

Listing 1: Timeout en AWS SDK v3

```
1 import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
2 import { S3Client } from '@aws-sdk/client-s3';
3
4 const dynamoClient = new DynamoDBClient({
5   region: process.env.AWS_REGION,
6   requestHandler: {
```

```

7     requestTimeout: 5_000,      // 5 segundos para DynamoDB
8     connectionTimeout: 2_000,  // 2 segundos para TCP handshake
9 },
10 });
11
12 const s3Client = new S3Client({
13     region: process.env.AWS_REGION,
14     requestHandler: {
15         requestTimeout: 10_000,  // 10 segundos para S3
16         connectionTimeout: 3_000,
17     },
18 });

```

Listing 2: Timeout en OCI SDK

```

1 import { GenerativeAiInferenceClient } from 'oci-generativeaiinference';
2
3 const genaiClient = new GenerativeAiInferenceClient({
4     authenticationDetailsProvider: new
5         DefaultConfigAuthenticationDetailsProvider(),
6     region: process.env.OCI_REGION ?? 'us-chicago-1',
7 });
8 // Timeout via AbortController
9 const controller = new AbortController();
10 const timeoutId = setTimeout(() => controller.abort(), 30_000);
11
12 try {
13     const response = await genaiClient.generateText(
14         { generateTextDetails },
15         { abortSignal: controller.signal }
16     );
17 } finally {
18     clearTimeout(timeoutId);
19 }

```

2.2 Timeouts por servicio

Servicio	Operación	Timeout	Justificación
DynamoDB	GetItem, PutItem, Query	5s	Operación simple, debe ser rápida
S3	GetObject, PutObject	10s	Puede ser archivo grande
SQS	SendMessage, ReceiveMessage	10s	Confiable pero no instantáneo
EventBridge	PutEvents	5s	Operación simple
SNS	Publish	10s	Confiable
OCI GenAI	generateText	30s	LLM puede tardar en razonar
OCI Document AI	analyzeDocument	60s	PDFs grandes tardan más
SES (email)	SendEmail	10s	Confiable

3 Circuit Breaker con opossum

3.1 Qué es y por qué lo necesitamos

Un circuit breaker es un patrón que previene llamadas a un servicio que está fallando. Funciona como un interruptor eléctrico:

- **CLOSED** (cerrado): llamadas pasan normalmente
- **OPEN** (abierto): llamadas se rechazan inmediatamente (fail-fast)
- **HALF_OPEN**: tras un tiempo, se permite UNA llamada de prueba

Si OCI GenAI empieza a fallar (50 % de errores), **no queremos seguir gastando tiempo y dinero** en llamadas que van a fallar. El circuit breaker las rechaza al instante, permitiéndonos caer al siguiente modelo de la cadena de fallback.

3.2 Implementación con opossum

Listing 3: Circuit breaker para Cohere Command R

```

1 import CircuitBreaker from 'opossum';
2 import { ociGenai } from './ociGenai';
3 import { logger } from '../handlers/_shared/logger';
4
5 interface GenerateArgs {
6   prompt: string;
7   systemPrompt?: string;
8   maxTokens?: number;
9   temperature?: number;
10 }
11
12 async function callCohere(args: GenerateArgs): Promise<string> {
13   return ociGenai.generateText(args.prompt, {
14     model: 'cohere.command-r',
15     systemPrompt: args.systemPrompt,
16     maxTokens: args.maxTokens,
17     temperature: args.temperature,
18   });
19 }
20
21 const breaker = new CircuitBreaker(callCohere, {
22   timeout: 30_000, // 30 segundos max por llamada
23   errorThresholdPercentage: 50, // 50% de errores -> abre
24   resetTimeout: 30_000, // 30 segundos en estado OPEN antes de
    probar
25   rollingCountTimeout: 10_000, // Ventana de 10s para calcular %
26   rollingCountBuckets: 10, // 10 buckets en la ventana
27   volumeThreshold: 5, // Minimo 5 calls antes de evaluar
28   name: 'cohere-command-r',
29 });
30
31 // Eventos para observabilidad
32 breaker.on('open', () => logger.warn('Circuit OPEN - OCI Cohere', {
33   breaker: 'cohere' }));
34 breaker.on('halfOpen', () => logger.info('Circuit HALF_OPEN - probando
    OCI Cohere'));
35 breaker.on('close', () => logger.info('Circuit CLOSED - OCI Cohere
    recuperado'));
36 breaker.on('reject', () => logger.warn('Call rechazada por circuit
    abierto'));

```

```

36 breaker.on('timeout', () => logger.error('Timeout en OCI Cohere'));
37 breaker.on('failure', (err) => logger.error('Fallo en OCI Cohere', {
    error: String(err) }));
38
39 export async function callCohereSafe(args: GenerateArgs): Promise<string>
40 > {
41   return breaker.fire(args);
42 }

```

3.3 Métricas del circuit breaker

Listing 4: Exponer metricas del circuit breaker

```

1 // En CloudWatch custom metrics
2 import { CloudWatch } from '@aws-sdk/client-cloudwatch';
3
4 setInterval(() => {
5   const cw = new CloudWatch({});
6   cw.putMetricData({
7     Namespace: 'SentinelAcademia/CircuitBreaker',
8     MetricData: [
9       {
10        MetricName: 'State',
11        Value: breaker.opened ? 1 : 0,
12        Unit: 'None',
13        Dimensions: [{ Name: 'Breaker', Value: 'cohere-command-r' }],
14      },
15      {
16        MetricName: 'Failures',
17        Value: breaker.stats.failures,
18        Unit: 'Count',
19        Dimensions: [{ Name: 'Breaker', Value: 'cohere-command-r' }],
20      },
21      {
22        MetricName: 'Successes',
23        Value: breaker.stats.successes,
24        Unit: 'Count',
25        Dimensions: [{ Name: 'Breaker', Value: 'cohere-command-r' }],
26      },
27    ],
28  });
29 }, 60_000); // Cada 60 segundos

```

4 Fallback Chain Multi-Modelo

4.1 Cadena de modelos

El sistema intenta los modelos en orden de **costo-beneficio**:

1. **Cohere Command R** (primario): \$0.50 input / \$1.50 output por 1M tokens
2. **Llama 3.1 70B** (secundario): \$0.72 input / \$0.72 output por 1M tokens
3. **Cohere Command R+** (terciario): \$3.00 input / \$15.00 output por 1M tokens
4. **Respuesta controlada** (degradado): mensaje fijo sin LLM

Decisión de costos

Usar Command R+ como terciario (no primario) reduce costos en ~70% vs. una arquitectura que lo usara siempre, mientras mantiene calidad cuando los modelos baratos fallan.

4.2 Implementación del router con fallback

Listing 5: LLM Router con fallback chain

```

1 import { callCohereSafe } from './breakers/cohereBreaker';
2 import { callLlamaSafe } from './breakers/llamaBreaker';
3 import { callCommandPlusSafe } from './breakers/commandPlusBreaker';
4 import { logger } from '../handlers/_shared/logger';
5
6 export interface GenerateArgs {
7   prompt: string;
8   systemPrompt?: string;
9   maxTokens?: number;
10  temperature?: number;
11 }
12
13 export async function generateWithFallback(args: GenerateArgs): Promise<
14   string> {
15   const models = [
16     { name: 'cohere.command-r', fn: () => callCohereSafe(args) },
17     { name: 'meta.llama-3.1-70b-instruct', fn: () => callLlamaSafe(args) },
18     { name: 'cohere.command-r-plus', fn: () => callCommandPlusSafe(args) },
19   ];
20
21   for (const model of models) {
22     try {
23       const result = await model.fn();
24       logger.info('LLM call exitosa', { model: model.name, length:
25         result.length });
26       return result;
27     } catch (err) {
28       logger.error('LLM call fallo, intentando siguiente', {
29         model: model.name,
30         error: String(err),
31       });
32       // Continuar al siguiente modelo
33     }
34   }
35
36   // Todos los modelos fallaron: respuesta controlada
37   logger.error('Todos los modelos LLM fallaron, usando respuesta
38     controlada');
39   return safeDefaultResponse();
40 }
41
42 function safeDefaultResponse(): string {
43   return JSON.stringify({
44     categoria: 'OTRA',
45     subcategoria: 'NO_CLASIFICADA',
  
```

```

43   criticidad: 'MEDIA',
44   criticidadJustificacion: 'Sistema no disponible para clasificar
    automaticamente. Requiere revision manual.',
45   sentimiento: 'NEUTRO',
46   entidades: [],
47   temasClave: [],
48   accionSugerida: 'Revision manual por equipo de bienestar',
49   prioridad: 5,
50   requiereNotificacionInmediata: false,
51   metadata: {
52     analisisFallback: true,
53     timestamp: new Date().toISOString(),
54   },
55   });
56 }

```

4.3 Justificación de cada modelo

Modelo	Costo/1M in	Costo/1M out	Cuándo usarlo
cohere.command-r	\$0.50	\$1.50	Primary. Excelente en JSON/structured output, barato, rápido. Suficiente para >80% de los casos.
meta.llama-3.1-70b-instruct	\$0.72	\$0.72	Fallback 1. Open weights, robusto, mismo precio input/output. Bueno para chat y RAG.
cohere.command-r-plus	\$3.00	\$15.00	Fallback 2. Reservado para casos extremos. 5x más caro que R. Razonamiento profundo.
Respuesta controlada	\$0	\$0	Degradado. Garantiza que el sistema NUNCA falla por el LLM. Marca para revisión manual.

5 Rate Limiting con Token Bucket

5.1 Algoritmo y justificación

Implementamos **token bucket** mediante la librería `rate-limiter-flexible`. Cada usuario tiene una "balde" que se rellena con tokens a velocidad constante. Cada request consume un token. Si no hay tokens, se rechaza con HTTP 429.

Listing 6: Rate limiter por usuario y por IP

```

1  import { RateLimiterMemory, RateLimiterRedis } from 'rate-limiter-
    flexible';
2
3  // En memoria (hackathon) o Redis (produccion)
4  const userLimiter = new RateLimiterMemory({
5    points: 100,           // 100 requests

```



```

6   duration: 60,           // por 60 segundos
7   blockDuration: 60,      // bloqueo 60s si se excede
8 });
9
10  const ipLimiter = new RateLimiterMemory({
11    points: 200,           // 200 requests
12    duration: 60,          // por minuto
13  });
14
15  export async function checkRateLimit(userId: string, ip: string):
16    Promise<{
17    allowed: boolean;
18    retryAfter?: number;
19  }> {
20    try {
21      await userLimiter.consume(userId, 1);
22      await ipLimiter.consume(ip, 1);
23      return { allowed: true };
24    } catch (err) {
25      if (err instanceof Error) {
26        return { allowed: false, retryAfter: 60 };
27      }
28      return { allowed: false };
29    }
30  }

```

5.2 Integración en handler

Listing 7: Rate limit en Lambda API Gateway

```

1  import { checkRateLimit } from './_shared/rateLimiter';
2
3  export const handler = withErrorHandling(async (event) => {
4    const userId = event.requestContext.authorizer?.jwt?.claims?.sub ?? '
5      anonymous';
6    const ip = event.requestContext.http?.sourceIp ?? '0.0.0.0';
7
8    const rate = await checkRateLimit(userId, ip);
9    if (!rate.allowed) {
10     return {
11       statusCode: 429,
12       headers: {
13         'Retry-After': String(rate.retryAfter),
14         'x-correlation-id': getCorrelationId(event),
15       },
16       body: JSON.stringify({
17         code: 'RATE_LIMITED',
18         message: 'Demasiadas solicitudes. Intenta en 60 segundos.',
19         correlationId: getCorrelationId(event),
20       }),
21     };
22   }
23   // Continuar con la logica normal...
24 });

```

5.3 Configuración por endpoint

Endpoint	Límite	Razón
POST /api/quejas	5/min por usuario	Evita spam de quejas duplicadas
GET /api/quejas	60/min por usuario	Lectura es barata
POST /api/chat	20/min por usuario	LLM es caro
GET /api/dashboard/metrics	30/min por usuario	Cache debería estar activo

6 Retry con Exponential Backoff y Jitter

6.1 Cuándo reintentar

Sí reintentar:

- Errores de red transitorios (5xx, ECONNRESET, ETIMEDOUT)
- Rate limits (429) con Retry-After
- Timeouts (superados pero no críticos)

NO reintentar:

- Errores 4xx del cliente (validacion, auth)
- Errores de programacion (TypeError, etc)
- Después de 3 intentos

6.2 Implementación con p-retry

Listing 8: Retry con exponential backoff

```

1 import pRetry, { AbortError } from 'p-retry';
2
3 async function callWithRetry<T>(  
4   fn: () => Promise<T>,  
5   options: { retries?: number; minTimeout?: number; maxTimeout?: number  
6     } = {}  
7 ): Promise<T> {  
8   return pRetry(fn, {  
9     retries: options.retries ?? 3,  
10    minTimeout: options.minTimeout ?? 1000,    // 1 segundo  
11    maxTimeout: options.maxTimeout ?? 10_000,  // 10 segundos  
12    factor: 2,                                // Backoff exponencial  
13    randomize: true,                           // Jitter  
14    onFailedAttempt: (error) => {  
15      logger.warn('Reintento', {  
16        attempt: error.attemptNumber,  
17        retriesLeft: error.retriesLeft,  
18        error: String(error),  
19      });  
20    },  
21  });

```

```

22
23 // Uso
24 const result = await callWithRetry(() => callCohereSafe(args), { retries
    : 3 });

```

6.3 Backoff exponencial + jitter

El jitter evita "thundering herd"(todos reintentan al mismo tiempo):

Listing 9: Calculo del delay con jitter

```

1 function calculateDelay(attempt: number, baseMs: number = 1000): number
  {
2   // Exponential: 1s, 2s, 4s, 8s, 16s
3   const exponential = baseMs * Math.pow(2, attempt);
4   // Cap at 30 seconds
5   const capped = Math.min(exponential, 30_000);
6   // Jitter: +/- 25% del valor
7   const jitter = capped * (0.75 + Math.random() * 0.5);
8   return Math.floor(jitter);
9 }
10
11 // Ejemplo de delays: 750ms, 2200ms, 3100ms, 8500ms

```

7 Idempotencia Exactly-Once Processing

7.1 El problema

SQS garantiza **at-least-once delivery**: un mensaje puede entregarse más de una vez. Si nuestra Lambda processQueja procesa el mismo mensaje 2 veces, el LLM se llama 2 veces (gastamos dinero) y se duplica el análisis en DynamoDB.

7.2 La solución: SHA-256 + conditional writes

Listing 10: Idempotencia con hash y conditional writes

```

1 import { createHash } from 'crypto';
2 import { docClient, TABLE_NAME } from '../handlers/_shared/dynamoClient';
3
4 ;
5 import { PutCommand } from '@aws-sdk/lib-dynamodb';
6
7 function computeIdempotencyKey(payload: unknown): string {
8   // Normalizar y hashear
9   const normalized = JSON.stringify(payload, Object.keys(payload).sort());
10   return createHash('sha256').update(normalized).digest('hex');
11 }
12
13 async function processQueja(message: QuejaProcessMessage): Promise<void>
14 {
15   const idempotencyKey = computeIdempotencyKey(message.payload);
16
17   // 1. Check si ya fue procesada
18   const existing = await docClient.send(new GetCommand({
19     TableName: TABLE_NAME,
20     Key: { pk: `IDEMPOTENCY#${idempotencyKey}`, sk: 'PROCESSED' },

```

```

18     }));
19
20     if (existing.Item) {
21         logger.info('Queja ya procesada, saltando', { idempotencyKey });
22         return; // ACK del mensaje
23     }
24
25     // 2. Procesar la queja con el LLM
26     const analysis = await analizarQueja(message.payload.input);
27
28     // 3. Transaccion atomica: guardar analisis + marcar como procesada
29     await docClient.send(new TransactWriteCommand({
30         TransactItems: [
31             {
32                 Put: {
33                     TableName: TABLE_NAME,
34                     Item: {
35                         pk: 'QUEJA#${message.payload.quejaId}',
36                         sk: 'ANALYSIS#' + new Date().toISOString(),
37                         ...analysis,
38                         quejaId: message.payload.quejaId,
39                         createdAt: new Date().toISOString(),
40                     },
41                 },
42             },
43             {
44                 Put: {
45                     TableName: TABLE_NAME,
46                     Item: {
47                         pk: 'IDEMPOTENCY#${idempotencyKey}',
48                         sk: 'PROCESSED',
49                         quejaId: message.payload.quejaId,
50                         processedAt: new Date().toISOString(),
51                         ttl: Math.floor(Date.now() / 1000) + (7 * 24 * 60 * 60), //
52                             7 dias
53                     },
54                     ConditionExpression: 'attribute_not_exists(pk)', // Atomic
55                             check
56                 },
57             },
58         ],
59     }));
60
61     logger.info('Queja procesada exitosamente', { quejaId: message.payload
62         .quejaId });
63 }

```

8 Dead Letter Queues (DLQ)

8.1 Obligatorias en cada cola

Toda cola SQS tiene una DLQ asociada. Después de 3 intentos fallidos, el mensaje va a la DLQ para inspección manual.

Listing 11: Report batch item failures

```

1 import { SQSEvent, SQSBatchResponse } from 'aws-lambda';

```

```

2
3 export const handler = async (event: SQSEvent): Promise<SQSBatchResponse
  > => {
4   const failures: { itemIdentifier: string }[] = [];
5
6   for (const record of event.Records) {
7     try {
8       const message = JSON.parse(record.body) as QuejaProcessMessage;
9       await processQueja(message);
10    } catch (err) {
11      logger.error('Error procesando mensaje', {
12        messageId: record.messageId,
13        error: String(err),
14      });
15      failures.push({ itemIdentifier: record.messageId });
16    }
17  }
18
19  return { batchItemFailures: failures };
20 };

```

8.2 SAM template para DLQ

Listing 12: SAM: SQS con DLQ obligatoria

```

1 ComplaintsQueue:
2   Type: AWS::SQS::Queue
3   Properties:
4     QueueName: complaints-queue
5     VisibilityTimeout: 180      # 6x el timeout de la Lambda
6     MessageRetentionPeriod: 1209600 # 14 días
7     RedrivePolicy:
8       deadLetterTargetArn: !GetAtt ComplaintsDLQ.Arn
9       maxReceiveCount: 3
10
11 ComplaintsDLQ:
12   Type: AWS::SQS::Queue
13   Properties:
14     QueueName: complaints-dlq
15     MessageRetentionPeriod: 1209600
16
17 # Alarma si la DLQ tiene mensajes
18 DLQAlarm:
19   Type: AWS::CloudWatch::Alarm
20   Properties:
21     AlarmName: complaints-dlq-not-empty
22     MetricName: ApproximateNumberOfMessagesVisible
23     Namespace: AWS/SQS
24     Dimensions:
25       - Name: QueueName
26         Value: !GetAtt ComplaintsDLQ.QueueName
27     Statistic: Sum
28     Period: 300
29     EvaluationPeriods: 1
30     Threshold: 0
31     ComparisonOperator: GreaterThanThreshold
32     AlarmActions:

```

33 - !Ref AlertTopic

9 Validación del Output del LLM con Zod

9.1 El problema de los LLMs

Los LLMs no siempre devuelven JSON válido. A veces:

- Agregan markdown de tipo “triple backtick json” alrededor del JSON
- Incluyen explicaciones antes/después
- Inventan campos extra
- Omiten campos requeridos
- Devuelven strings en campos que deberían ser números

9.2 Schema Zod estricto

Listing 13: Schema Zod para validacion

```

1 import { z } from 'zod';
2
3 export const AnalisisSchema = z.object({
4   categoria: z.enum([
5     'ACADEMICA', 'INFRAESTRUCTURA', 'ACOSO',
6     'ADMINISTRATIVA', 'SALUD', 'OTRA'
7   ]),
8   subcategoria: z.string().min(1).max(100),
9   criticidad: z.enum(['BAJA', 'MEDIA', 'ALTA', 'CRITICA']),
10  criticidadJustificacion: z.string().min(10).max(500),
11  sentimiento: z.enum(['POSITIVO', 'NEUTRO', 'NEGATIVO', '
    NEGATIVO_FUERTE']),
12  entidades: z.array(z.object({
13    tipo: z.string(),
14    texto: z.string().max(200),
15  })).max(20),
16  temasClave: z.array(z.string()).min(1).max(10),
17  accionSugerida: z.string().min(10).max(500),
18  prioridad: z.number().int().min(1).max(10),
19  requiereNotificacionInmediata: z.boolean(),
20 }).strict(); // Rechaza campos extra
21
22 export type Analisis = z.infer<typeof AnalisisSchema>;

```

9.3 Funciones de extracción y validación

Listing 14: Extraer JSON del output y validar

```

1 function extractJson(text: string): unknown {
2   // Intentar parsear directamente
3   try {
4     return JSON.parse(text);
5   } catch {
6     // Buscar bloque JSON en el texto

```

```

7     const match = text.match(/\{([\s\S]*\})/);
8     if (!match) {
9         throw new Error('No se encontro JSON en el output del LLM');
10    }
11    return JSON.parse(match[0]);
12  }
13 }
14
15 export async function analizarQueja(texto: string): Promise<Análisis> {
16     const raw = await generateWithFallback({
17         prompt: 'Analiza esta queja: "' + texto + '"',
18         systemPrompt: SYSTEM_PROMPT,
19         temperature: 0.3, // Bajo para consistencia
20         maxTokens: 800,
21     });
22
23     const json = extractJson(raw);
24     const parsed = AnalisisSchema.safeParse(json);
25
26     if (!parsed.success) {
27         logger.error('LLM output invalido', {
28             errors: parsed.error.flatten(),
29             rawOutput: raw.slice(0, 500),
30         });
31         throw new Error('Output invalido: ' + parsed.error.message);
32     }
33
34     return parsed.data;
35 }

```

10 Logging Estructurado

10.1 Formato JSON obligatorio

Todo log es JSON válido, parseable por CloudWatch Logs Insights.

Listing 15: Logger estructurado

```

1 type LogLevel = 'debug' | 'info' | 'warn' | 'error';
2
3 interface LogContext {
4     correlationId?: string;
5     userId?: string;
6     quejaId?: string;
7     modelo?: string;
8     tokens?: number;
9     latencyMs?: number;
10    [key: string]: unknown;
11 }
12
13 class StructuredLogger {
14     log(level: LogLevel, message: string, context: LogContext = {}) {
15         const entry = {
16             timestamp: new Date().toISOString(),
17             level: level.toUpperCase(),
18             message,
19             service: process.env.POWERTOOLS_SERVICE_NAME ?? 'sentinel-academia',

```

```

20     ...context,
21   };
22   console.log(JSON.stringify(entry));
23 }
24
25 debug(message: string, context?: LogContext) { this.log('debug',
26   message, context); }
27 info(message: string, context?: LogContext) { this.log('info', message
28   , context); }
29 warn(message: string, context?: LogContext) { this.log('warn', message
30   , context); }
31 error(message: string, context?: LogContext) { this.log('error',
32   message, context); }
33 }
34
35 export const logger = new StructuredLogger();

```

10.2 Correlation ID end-to-end

El correlationId se genera en API Gateway (o en el cliente), se pasa por headers HTTP, se incluye en mensajes SQS, y se logea en cada paso. Permite trazabilidad completa de una queja.

Listing 16: Propagacion del correlationId

```

1 // En el handler
2 const correlationId = event.headers['x-correlation-id'] ?? crypto.
3   randomUUID();
4 logger.info('Lambda invoked', { correlationId });
5
6 // En el mensaje a SQS
7 await sqs.send(new SendMessageCommand({
8   QueueUrl: QUEUE_URL,
9   MessageBody: JSON.stringify({
10     payload: data,
11     metadata: { correlationId, attempts: 0 },
12   }),
13   MessageAttributes: {
14     correlationId: { DataType: 'String', StringValue: correlationId },
15   },
16 }));
17
18 // En el consumer
19 const correlationId = record.messageAttributes?.correlationId?.
20   stringValue
21   ?? JSON.parse(record.body).metadata.correlationId;
22 logger.info('Procesando mensaje', { correlationId });

```

11 Observabilidad End-to-End

11.1 CloudWatch Logs Insights queries

Queries utiles para debugging:

Listing 17: Errores en la ultima hora

```

1 fields @timestamp, level, message, correlationId, error
2 | filter level = "ERROR"

```



```

3 | stats count() as total by message
4 | sort total desc
5 | limit 20

```

Listing 18: Trace por correlationId específico

```

1 fields @timestamp, level, message, modelo, tokens, latencyMs
2 | filter correlationId = "abc-123-def"
3 | sort @timestamp asc

```

Listing 19: Latencia p95 de Lambdas

```

1 fields @timestamp, @duration
2 | filter @type = "REPORT"
3 | stats percentile(@duration, 95) as p95 by bin(5m)

```

Listing 20: Costos estimados de LLM

```

1 fields @timestamp, modelo, tokens
2 | filter message = "LLM call exitosa"
3 | stats sum(tokens) as totalTokens by modelo
4 | sort totalTokens desc

```

11.2 Langfuse para tracing del LLM

Langfuse proporciona tracing detallado de cada llamada al LLM:

Listing 21: Tracing con Langfuse

```

1 import { Langfuse } from 'langfuse';
2
3 const langfuse = new Langfuse({
4   publicKey: process.env.LANGFUSE_PUBLIC_KEY!,
5   secretKey: process.env.LANGFUSE_SECRET_KEY!,
6   baseUrl: process.env.LANGFUSE_HOST,
7 });
8
9 export async function generateWithFallbackTraced(args: GenerateArgs):
10   Promise<string> {
11   const trace = langfuse.trace({
12     name: 'generate-with-fallback',
13     metadata: { correlationId: args.correlationId },
14   });
15   // ... logica de fallback ...
16
17   for (const model of models) {
18     const span = trace.span({
19       name: `llm-call-${model.name}`,
20       input: { prompt: args.prompt.slice(0, 200) },
21     });
22
23     try {
24       const result = await model.fn();
25       const tokens = estimateTokens(args.prompt + result);
26
27       span.end({
28         output: { text: result.slice(0, 200) },

```

```

29     usage: { total: tokens },
30     metadata: { latencyMs: Date.now() - start, model: model.name },
31   });
32
33   await langfuse.flushAsync();
34   return result;
35 } catch (err) {
36   span.end({ level: 'ERROR', statusMessage: String(err) });
37 }
38 }
39
40 await langfuse.flushAsync();
41 return safeDefaultResponse();
42 }

```

12 Costos y Monitoreo de LLM

12.1 Tracking de tokens

Listing 22: Estimar tokens consumidos

```

1 function estimateTokens(text: string): number {
2   // 1 token ~= 4 caracteres en español
3   return Math.ceil(text.length / 4);
4 }
5
6 // En el router
7 const inputTokens = estimateTokens(args.prompt + (args.systemPrompt ?? '
8   '));
9 const outputTokens = estimateTokens(result);
10 const cost = estimateCost(model.name, inputTokens, outputTokens);
11
12 logger.info('LLM call exitosa', {
13   modelo: model.name,
14   inputTokens,
15   outputTokens,
16   costUsd: cost,
17   latencyMs: Date.now() - start,
18 });
19
20 // Enviar a CloudWatch custom metric
21 metrics.putMetric('LlmTokensInput', inputTokens, Unit.Count, { modelo:
22   model.name });
23 metrics.putMetric('LlmTokensOutput', outputTokens, Unit.Count, { modelo:
24   model.name });
25 metrics.putMetric('LlmCost', cost, Unit.None, { modelo: model.name });

```

12.2 Estimacion de costos

Modelo	Input \$/1M	Output \$/1M	Uso en Sentinel
cohere.command-r	\$0.50	\$1.50	80 % (primary)
meta.llama-3.1-70b-instruct	\$0.72	\$0.72	15 % (fallback)
cohere.command-r-plus	\$3.00	\$15.00	4 % (extremo)

Modelo	Input \$/1M	Output \$/1M	Uso en Sentinel
Respuesta controlada	\$0	\$0	1 % (degradado)

Costo por queja promedio: ~\$0.0001 USD.

Proyección hackatón (500 quejas): ~\$0.05 USD total.

Proyección mensual (5000 quejas): ~\$0.50 USD.

13 Diagrama de Flujo Completo de Resiliencia

14 Resultados Esperados y SLAs

14.1 SLAs objetivo

Métrica	Objetivo	Medición
Tasa de éxito en procesamiento	$\geq 99\%$	CloudWatch Metric: ProcessSuccess / ProcessTotal
Tiempo de procesamiento (p95)	$< 30s$	CloudWatch Metric: ProcessingDuration
Latencia del LLM (p95)	$< 20s$	Custom Metric: LlmLatency
Disponibilidad del sistema	$\geq 99.5\%$	CloudWatch Synthetics Canary
Detección de caso crítico	$< 60s$ end-to-end	Tiempo entre POST /quejas y SNS publish
Tasa de falsos positivos (criticidad CRITICA)	$< 10\%$	Análisis manual semanal
DLQ vacía	100 % del tiempo	CloudWatch Alarm
Circuit breaker cerrado	$\geq 95\%$ del tiempo	Custom Metric: CircuitState

14.2 Cumplimiento de la rúbrica

- **Circuit breaker en TODA llamada a LLM:** opossum configurado por modelo
- **Fallback entre 2+ modelos:** cadena de 3 modelos + respuesta controlada
- **Rate limiting:** token bucket por usuario e IP
- **Retry con exponential backoff:** p-retry con jitter
- **DLQ en TODA cola SQS:** redrive policy + alarma
- **Idempotencia en consumers:** SHA-256 + conditional writes en DynamoDB
- **Timeouts en TODA llamada externa:** configurados por servicio
- **Logging estructurado con correlationId:** propagado end-to-end
- **Prompt estructurado con validación Zod:** schema estricto + safeParse
- **Monitoreo:** CloudWatch + X-Ray + Langfuse + alarms

*“La resiliencia no es un lujo, es un requisito.
Especialmente cuando se trata de bienestar de personas.”*

